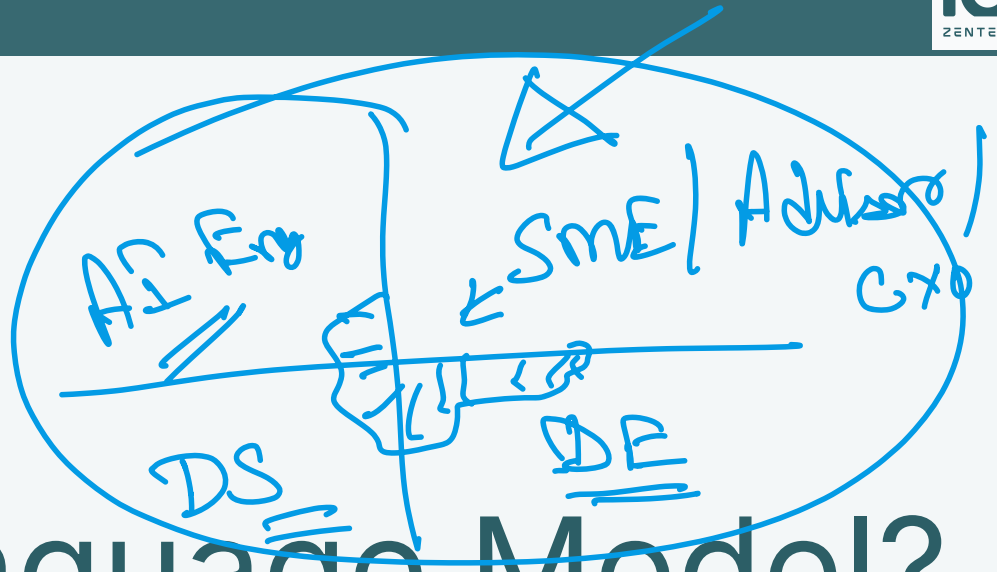
AhamX Bodhi

Where I Becomes Infinite



What is a Language Model?

Module 1.2: LLM Fundamentals | Prof. Sashikumaar Ganesan

1:30

Session Overview

What You Will Learn

- Core definition of a language model
- How LMs represent and process language
- Probability distributions over text
- The foundation of modern LLMs

Prerequisites

- Basic familiarity with AI concepts
- High-school level mathematics
- No prior ML experience required

Learning Objectives

By the End of This Session You Will Be Able To

1. **Define** a language model and articulate its core purpose (B1)
2. **Explain** how a language model assigns probabilities to text sequences (B2)
3. **Distinguish** between traditional NLP approaches and neural language models (B2)
4. **Describe** the relationship between language models and modern LLMs (B2)
5. **Identify** everyday applications powered by language models (B2)

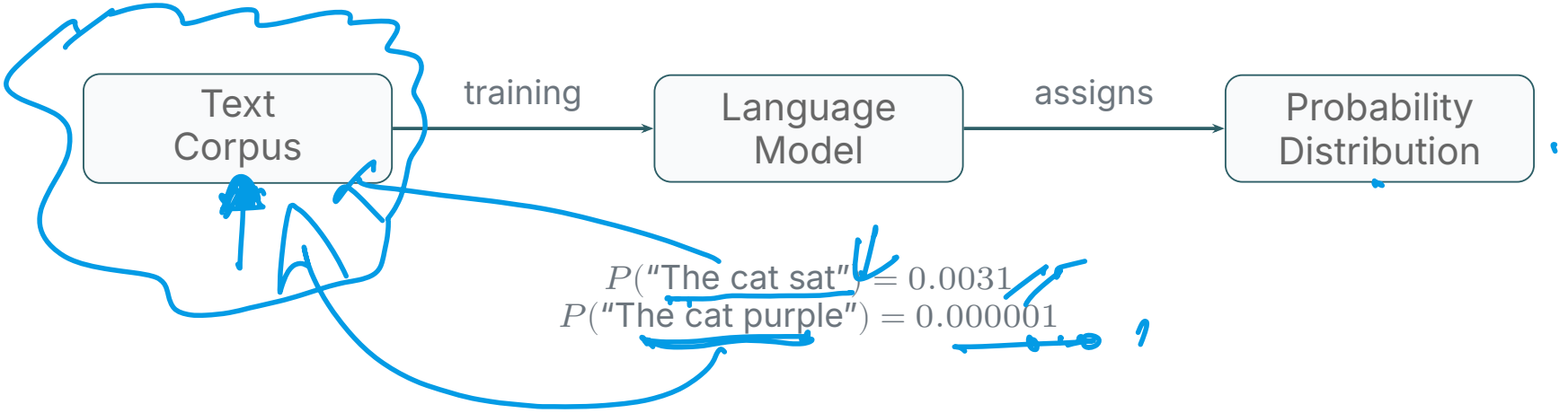
What is a Language Model?

Core Definition

- A **language model** is a probabilistic system that assigns a probability to sequences of words (or tokens)
- It captures statistical regularities in human language from large text corpora
- Given a sequence of words, it estimates: *how likely is this text?*

Formally, a language model defines a probability distribution $P(w_1, w_2, \ldots, w_n)$ over sequences of tokens. This single idea underpins everything from autocomplete to ChatGPT.

Language Model --- Core Idea



Key Intuition

Language models score text plausibility — likely sentences get high probability, unlikely ones get low probability.

Probability Over Text Sequences

The Chain Rule of Probability

- A language model decomposes joint probability using the **chain rule**:

$$P(w_1, \ldots, w_n) = \prod_{i=1}^{n} P(w_i \mid w_1, \ldots, w_{i-1})$$

- At each step, the model predicts the **next token** given all previous tokens
- This is the key mechanism behind autoregressive text generation

Example: $P(\text{“The cat sat on the mat”})$

$$= P(\text{The}) \cdot P(\text{cat} \mid \text{The}) \cdot P(\text{sat} \mid \text{The cat}) \cdots$$

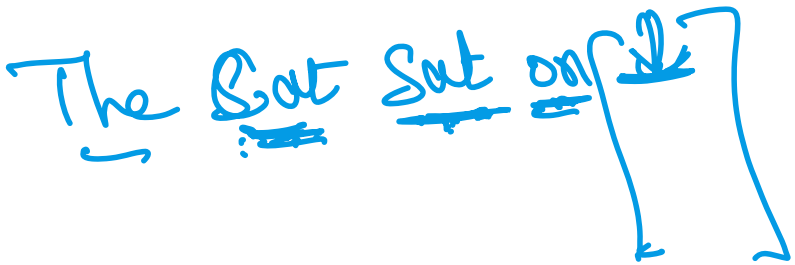
Evolution: From N-grams to Neural Language Models

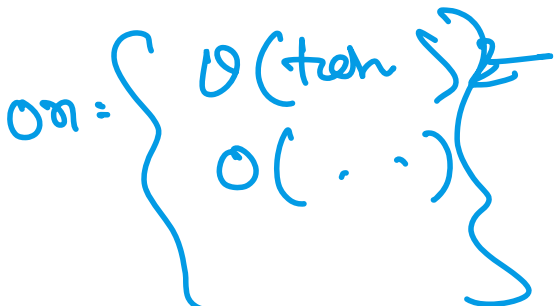
Traditional N-gram Models

- Count word co-occurrences in a corpus
- Estimate $P(w_i \mid w_{i-n+1}, \ldots, w_{i-1})$
- Simple, fast, interpretable
- Struggle with long-range dependencies
- Require huge memory for large n

Neural Language Models

- Learn continuous word representations (embeddings)
- Model long-range context via attention
- Generalize far beyond seen examples
- Scale to billions of parameters
- Power modern LLMs (GPT, Llama, Claude)





What Does a Language Model Actually Learn?

Implicit Knowledge Captured During Training

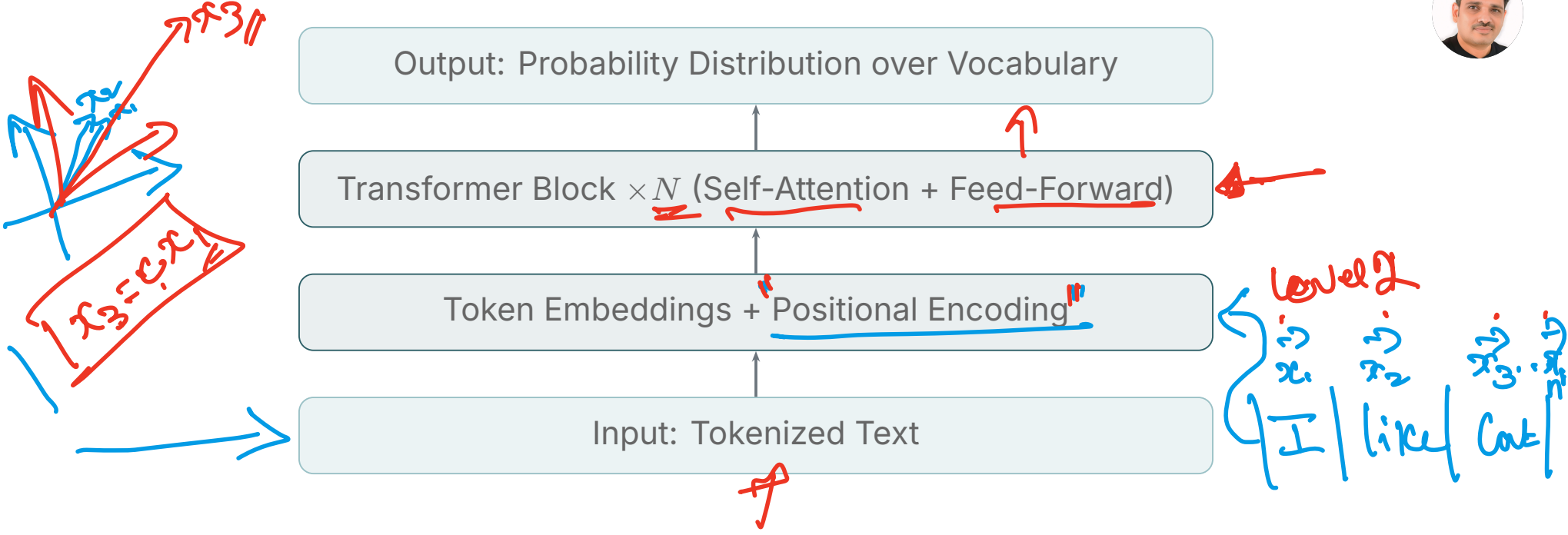
- **Syntax:** Grammatical sentence structure and word ordering
- **Semantics:** Word meanings, synonyms, analogies
- **World knowledge:** Facts, relationships, common sense
- **Style and tone:** Formal vs. informal, domain-specific language

The Remarkable Property

- No human labels needed — the training signal is the text itself
- Predicting the next word forces the model to understand language deeply
- This is called **self-supervised learning**

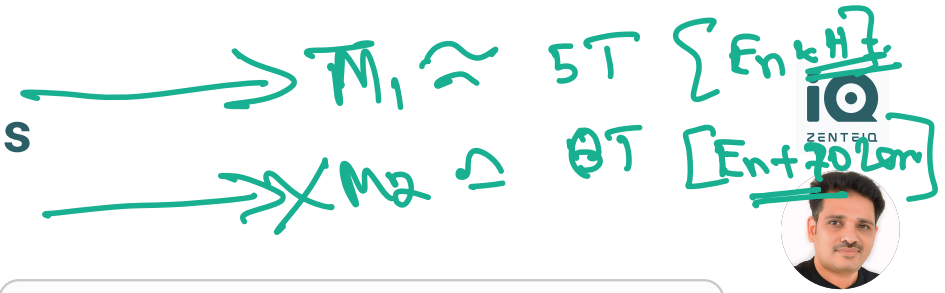
LLMs

Neural Language Model --- Architecture at a Glance



Each layer learns increasingly abstract representations — from raw tokens to rich semantic structure.

Tokens: The Basic Unit of Language Models

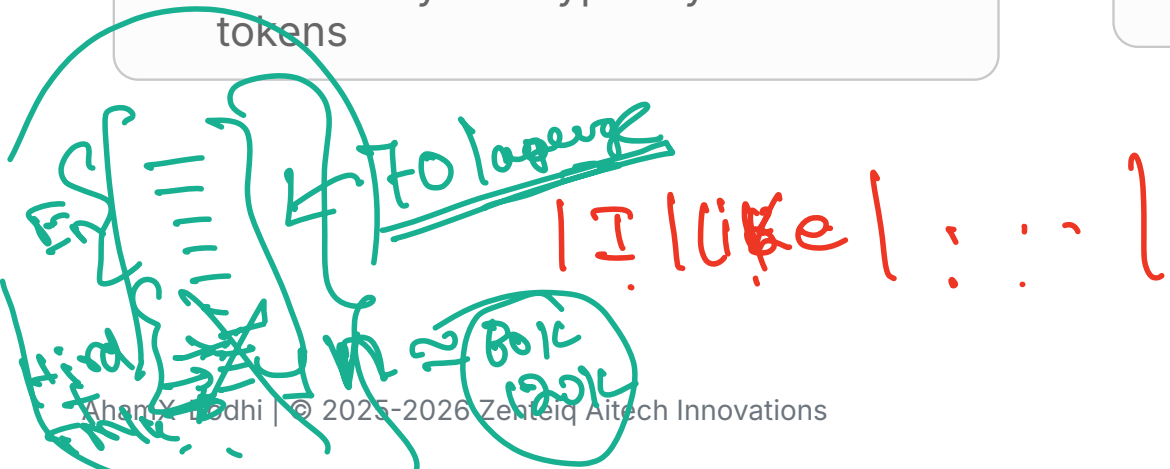


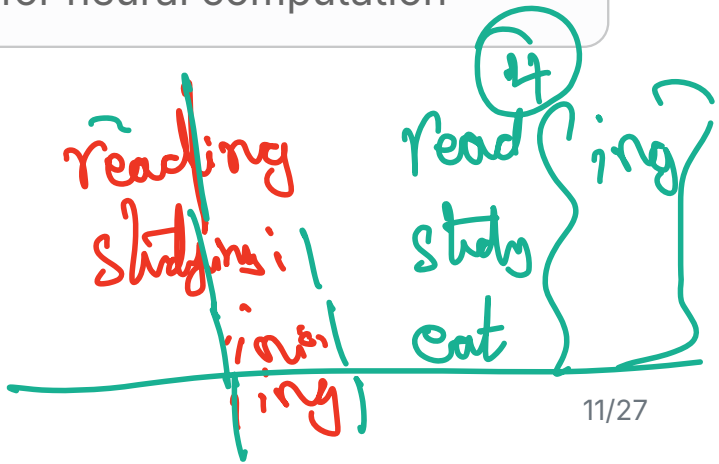
What is a Token?

- A token is a chunk of text — a word, sub-word, or character
- Modern LLMs use **sub-word tokenization** (BPE, WordPiece)
- “unbelievable” → [“un”, “believ”, “able”]
- Vocabulary size typically 32K–100K tokens

Why Sub-word Tokenization?

- Handles rare and unseen words gracefully
- Balances vocabulary size vs. sequence length
- Language-agnostic (works across languages)
- Efficient for neural computation





Training Objective: Next-Token Prediction

How Language Models Are Trained

- Given tokens $w_1, \ldots, w_{t-1}$, predict w_t
- Minimize **cross-entropy loss** (equivalently, maximize log-likelihood):

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(w_t \mid w_1, \ldots, w_{t-1}; \boldsymbol{\theta})$$

- Gradient descent updates parameters $\boldsymbol{\theta}$ to reduce loss

No labels needed — every sentence in the training corpus is both input and supervision. This enables training on *trillions* of tokens from the internet.

From Language Model to Large Language Model

What Makes an LLM "Large"?

- **Scale of parameters:** Billions to trillions of learnable weights
- **Scale of data:** Hundreds of billions to trillions of training tokens
- **Scale of compute:** Thousands of GPUs/TPUs for weeks or months
- **Emergent capabilities:** In-context learning, reasoning, instruction following

Scaling Laws (Kaplan et al., 2020)

- Performance improves predictably with more parameters, data, and compute
- Chinchilla scaling (Hoffmann et al., 2022) refined the optimal compute allocation
- Scaling is the core driver behind GPT-4, Llama 3, Gemini, and Claude

GenAI Application: Conversational AI

How Language Models Power Chatbots

- **Foundation:** A pre-trained LM provides broad world knowledge
- **Instruction tuning:** Fine-tuning on conversation data aligns the model to follow instructions
- **RLHF:** Reinforcement learning from human feedback improves helpfulness and safety
- **Deployment:** Served via API (OpenAI, Anthropic, Google)

Real Systems

- ChatGPT, Claude, Gemini, Llama 3 — all built on the same LM foundation
- Customer service bots, coding assistants, tutoring systems
- Multi-turn dialogue managed via the context window

GenAI Application: Code Generation

Why LMs Excel at Code

- Code is highly structured text — LMs capture syntax and semantics naturally
- Trained on billions of lines from GitHub, Stack Overflow, docs
- GitHub Copilot: real-time completion inside IDEs
- Claude and GPT-4 write, debug, explain code

Capabilities Unlocked

- Autocomplete entire functions
- Translate between programming languages
- Generate unit tests automatically
- Explain legacy codebases
- Fix bugs from natural language descriptions

GenAI Application: Document Intelligence

Summarization and Information Extraction

- Language models read, comprehend, and synthesize long documents
- Summarize legal contracts, medical records, research papers in seconds
- Extract structured data (dates, entities, clauses) from unstructured text

Industry Deployments

- **Legal tech:** Contract analysis, due diligence (Harvey AI, CoCounsel)
- **Healthcare:** Clinical note summarization, discharge summaries
- **Finance:** Earnings call analysis, financial report extraction
- **Enterprise:** Internal knowledge bases, HR policy Q&A

GenAI Application: Semantic Search

Traditional vs. Semantic Search

- Traditional: keyword matching (TF-IDF, BM25)
- Semantic: LM encodes meaning into dense vectors
- Finds conceptually related content, not just exact matches
- Handles paraphrases, synonyms, cross-lingual queries

Real-World Systems

- Google Search neural ranking
- Bing AI-powered results
- Enterprise search (Elastic, Pinecone)
- RAG pipelines for chatbots
- E-commerce product discovery

GenAI Application: Language Models Go Multimodal

Beyond Text: Vision-Language Models

- Language models serve as the reasoning backbone of multimodal systems
- Image captions, visual question answering, document understanding
- GPT-4 Vision, Gemini 1.5, Claude 3 — all extend LMs with vision encoders
- Audio and video understanding via modality-specific encoders feeding into LMs

Key Insight

The language model component remains central even in multimodal systems — images and audio are “translated” into token-like representations that the LM can process.

Skills You Are Building

Conceptual Skills

- Understanding probabilistic text modeling
- Distinguishing statistical vs. neural approaches
- Reasoning about model scale and capabilities
- Connecting LMs to real-world AI products

Technical Vocabulary

- Token, vocabulary, embedding
- Probability distribution, cross-entropy
- Autoregressive generation
- Pre-training, self-supervised learning
- Foundation model, LLM

Professional Roles That Use This Knowledge

Role	How Language Model Knowledge Is Applied
ML / AI Engineer	Select, deploy, and fine-tune LLMs for production systems
Data Scientist	Evaluate model performance, interpret outputs, design experiments
Research Scientist	Develop novel LM architectures and training procedures
Software Developer	Integrate LLM APIs into applications (chatbots, copilots)
Product Manager	Scope AI features, understand capability and limitations
AI Safety Engineer	Audit models for bias, hallucination, and safety risks

Knowledge Graph: Where This Concept Fits

What You Need Before This Concept

- **AI-GN-FN-TH-000001** — What is Generative AI? (foundational context)
- Basic understanding of probability (what does $P(A)$ mean?)
- Familiarity with the idea of a computer program processing text

Your Position in the Curriculum

- This is the **entry point** for all LLM-related concepts in the programme
- Module 1.2 bridges general AI awareness (M1.1) to technical depth (Modules 2–3)
- Completing this concept unlocks the rest of the LLM Fundamentals track

What This Concept Unlocks --- Part 1

Immediate Next Concepts in Module 1.2

- **AI-LM-FN-TH-000002** — Probability and Token Prediction (next step)
- **AI-LM-FN-TH-000003** — Autoregressive Generation (builds directly on this)
- **AI-LM-FN-TH-000004** — Model Sizing and Scaling Laws (extends scale discussion)
- **AI-LM-FN-TH-000005** — Foundation Models Overview (broadens scope)

What This Concept Unlocks --- Part 2

Downstream Concepts Across the Programme

- **AI-TR-FN-TH-000001** — What is a Transformer? (Module 2)
- **AI-TK-FN-TH-000001** — What is Tokenization? (Module 3)
- **AI-LM-IN-TH-000001** — API Fundamentals for LLMs (Module 3)
- **AI-FT-IN-TH-000001** — What is Fine-Tuning? (Module 7)

Common Misconceptions

Misconception

- “LLMs understand language like humans do”
- “A language model retrieves stored answers”
- “Bigger is always better for all tasks”
- “Language models are deterministic”

Correction

- They model statistical patterns, not cognition
- They *generate* based on learned distributions
- Small fine-tuned models often outperform large general ones
- Sampling temperature introduces controlled randomness

Key Takeaways

What We Covered Today

- A **language model** assigns probability distributions over sequences of tokens
- It learns from raw text via **self-supervised next-token prediction**
- Neural LMs use embeddings + transformer layers to capture deep linguistic structure
- **Scaling** parameters, data, and compute yields Large Language Models
- LMs are the foundation of conversational AI, code generation, document intelligence, and semantic search

Next Steps

Immediate Actions

- Complete the self-assessment quiz for Module 1.2
- Explore the next concept: Probability and Token Prediction
- Interact with an LLM API (OpenAI, Anthropic) — observe token-by-token generation

Recommended Reading

- Jurafsky & Martin, *Speech and Language Processing* Ch. 3 (N-gram LMs)
- Vaswani et al. (2017), “Attention Is All You Need”
- Kaplan et al. (2020), “Scaling Laws for Neural Language Models”

Thank You

What is a Language Model? | Module 1.2: LLM Fundamentals